

UNIDAD 9 · SEMANAS 15–16

SO nativos de IA: agentes y LLM en el sistema operativo

📅 Semanas 15–16 · fechas por confirmar </> Laboratorio TD8 (resuelto + desafío IA)

~ 4 animaciones

MANUAL M9 · TEORÍA DE LA UNIDAD

SO nativos de IA: agentes y LLM en el sistema operativo

Lectura base de la unidad 9 (≈ 1.5 h). Esta unidad junta todo el curso: un agente de IA es un proceso más, que usa llamadas al sistema, archivos, memoria, hilos y sockets, pero cuyas decisiones las toma un modelo que puede equivocarse. **Los productos citados cambian rápido:** cada afirmación lleva su fuente y fecha (consultadas el 25/09/2026).

CONTENIDO

- | | | | |
|----|--|----|--|
| §1 | Del chat al agente | §4 | Las primitivas de aislamiento de Linux |
| §2 | Qué anunciaron Windows y Linux | §5 | El modelo propone, el SO dispone |
| §3 | La inferencia como carga de trabajo del SO | | |

§1 Del chat al agente

Un modelo de lenguaje (LLM) recibe texto y predice el texto siguiente. Un **agente** agrega un bucle: el modelo propone una *acción* —una llamada a una herramienta, como `read_file("notas.txt")`—, un programa la ejecuta y el resultado vuelve al modelo. Así un asistente puede ordenar archivos, cambiar la configuración o escribir en una aplicación.

Para el sistema operativo esto es nuevo por dos razones: el agente actúa **en nombre del usuario pero no es el usuario**, y sus decisiones provienen de un modelo estadístico que puede equivocarse o ser manipulado (por ejemplo, con instrucciones escondidas en un documento que lee: la *inyección de instrucciones*). Por eso las preguntas de la unidad son de sistemas operativos: ¿quién es el agente?, ¿qué puede tocar?, ¿quién autoriza?, ¿cómo se audita?

§2 Qué anunciaron Windows y Linux

Windows

- **Agent workspace:** un espacio separado donde el agente trabaja en paralelo al usuario, con **su propia cuenta**; accede a las carpetas conocidas (Documentos, Descargas, Escritorio, Música, Imágenes, Videos) solo si el usuario lo habilita y consiente. Se activa en *Configuración* → *Sistema* → *Componentes de IA* → *Funciones agénticas experimentales* (vista previa). Fuente: Microsoft Support, "Experimental agentic features".
- **Agent connectors:** servidores del *Model Context Protocol* (MCP) que conectan al agente con aplicaciones y herramientas del sistema, registrados en el **On-Device Registry**; los primeros, para el Explorador de archivos y Configuración (Insiders, diciembre de 2025). Fuentes: Microsoft Support; The Register, 08/12/2025.
- Principios declarados: acciones del agente **observables y distinguibles** de las del usuario, **autorización** del usuario para datos y acciones, y **mínimo privilegio** (nunca más permisos que el usuario que lo inició).
- **Microsoft Execution Containers** (Build 2026, 2 de junio de 2026): una capa de ejecución con políticas declarativas para agentes en Windows y WSL, con aislamiento por *proceso*, por *sesión* o en una *micro-VM* respaldada por el hipervisor; en vista previa temprana. Fuente: Windows Developer Blog, "Windows platform security for AI agents", 02/06/2026.

Linux

- No hay un "agente del sistema" único: el **núcleo aporta primitivas** (§4) y las distribuciones y aplicaciones las combinan.
- **Canonical (Ubuntu)** anunció en 2026 funciones de IA centradas en la inferencia local con modelos abiertos (*inference snaps*) y agentes confinados con los mecanismos de los snaps; las primeras vistas previas se esperan en Ubuntu 26.10. No habrá un interruptor global de IA: se desactiva quitando los snaps. Fuentes: OMG! Ubuntu, 04/2026; Phoronix.
- Para aceleradores de IA, Linux tiene el subsistema `accel` (controladores de NPU de Intel y AMD). En la comunidad del núcleo, desde diciembre de 2025 la documentación pide declarar la ayuda de herramientas de IA con la etiqueta `Assisted-by`; la firma `Signed-off-by` solo puede ponerla una persona.

Practica esto → animación A9.1 · Arquitectura de un SO agéntico

§3 La inferencia como carga de trabajo del SO

Ejecutar un modelo es, para el sistema operativo, un programa con un perfil muy particular:

- **Pesos enormes y de solo lectura.** Se cargan con `mmap` (unidad 6): no se copian, se leen desde la caché de páginas al tocarlos (fallos de página) y varios procesos comparten las mismas páginas físicas. El micro-LLM del TD8 mapea sus $\approx 0,7$ MB de pesos así.
- **La caché KV.** Por cada token ya procesado, el modelo guarda claves y valores de cada capa para no recalcularlos: memoria que crece con la longitud de la conversación (en el TD8, $2 \times$

capas × contexto × 64 × 4 bytes).

- **Dos fases.** El *prefill* procesa toda la petición de una vez; el *decode* genera un token por paso, cada uno leyendo todos los pesos.
- **Aceleradores.** GPU y NPU se manejan como dispositivos: controladores, memoria propia y planificación del acceso entre procesos.

Practica esto → animación A9.2 · Dentro del micro-LLM

§4 Las primitivas de aislamiento de Linux

PRIMITIVA	QUÉ LIMITA	NECESITA ROOT
Usuarios y permisos	Archivos según dueño, grupo y modo	Para crear usuarios
Landlock (desde 5.13)	Qué rutas del sistema de archivos puede abrir el proceso; puertos TCP (ABI 4); señales y sockets abstractos fuera de su ámbito (ABI 6)	No: el proceso se restringe a sí mismo
seccomp	Qué llamadas al sistema puede hacer	No (con <code>no_new_privs</code>)
Espacios de nombres	Qué ve: procesos, montajes, red, usuarios (base de los contenedores)	No, con espacios de usuario
cgroups	Cuánto consume: CPU, memoria, E/S	Normalmente sí (o delegado por systemd)
<code>setrlimit</code>	Límites por proceso: tiempo de CPU, tamaño de archivos, descriptores	No, para bajarlos

El TD8 usa Landlock, `setrlimit` y `alarm` en `agent_exec`: después de `landlock_restrict_self` el proceso ya no puede salir del espacio de trabajo, ni aunque su propio código lo intente. La restricción se hereda y no se puede deshacer.

§5 El modelo propone, el SO dispone

La lección central de la unidad: **la salida del modelo es texto no confiable**. El SO agéntico del TD8 la trata como cualquier entrada externa:

1. **Parser estricto:** solo `nombre("arg", "arg")` de un catálogo cerrado; todo lo demás se rechaza.
2. **Política:** rutas relativas dentro del espacio de trabajo, sin `..` y resolviendo enlaces simbólicos (`realpath`).

3. **Consentimiento** del usuario para lo destructivo.
 4. **Sandbox del núcleo** (Landlock) como última línea: si la política falla o se desactiva, el núcleo sigue negando el acceso.
 5. **Auditoría**: cada decisión queda en `audit.log` con el uid y pid del cliente (`SO_PEERCREATED`).
- Es el principio de *defensa en profundidad*: ninguna capa confía en que la anterior no falló.

Practica esto → animación A9.4 · El modelo propone, el SO dispone, y el TD8